Promises in JavaScript

JavaScript

Contents

🔹 Introduction

JavaScript is single-threaded, meaning it executes one operation at a time. But real-world applications often require waiting for tasks like API responses or file loading. Promises make handling these asynchronous operations cleaner and more efficient than traditional callbacks.

What is a Promise?

A **Promise** in JavaScript is an object that represents the eventual success (resolved) or failure (rejected) of an asynchronous operation. It acts as a placeholder for a value that isn't available yet.

It has three states:

1. **Pending** – The initial state, neither fulfilled nor rejected.
2. **Fulfilled** – The operation completed successfully.
3. **Rejected** – The operation failed.

Think of a Promise like ordering pizza:

- You **place the order** (Promise is created – Pending).
- The pizza arrives successfully (Promise is **fulfilled** – `resolve`).
- The order fails because they ran out of dough (Promise is **rejected** – `reject`).

Creating a Promise

An instance of a promise object can be created with the syntax:

```
new Promise((resolve, reject) {
//executor function
})
```

The **Promise** constructor takes a single argument - an executor function, which itself takes two parameters: resolve and reject.

Both resolve and reject are callback functions which are automatically passed as arguments to your executor function that you call inside the executor to settle the promise.

The first code snippet used an anonymous arrow function as the executor function but we can use a named function too like this:

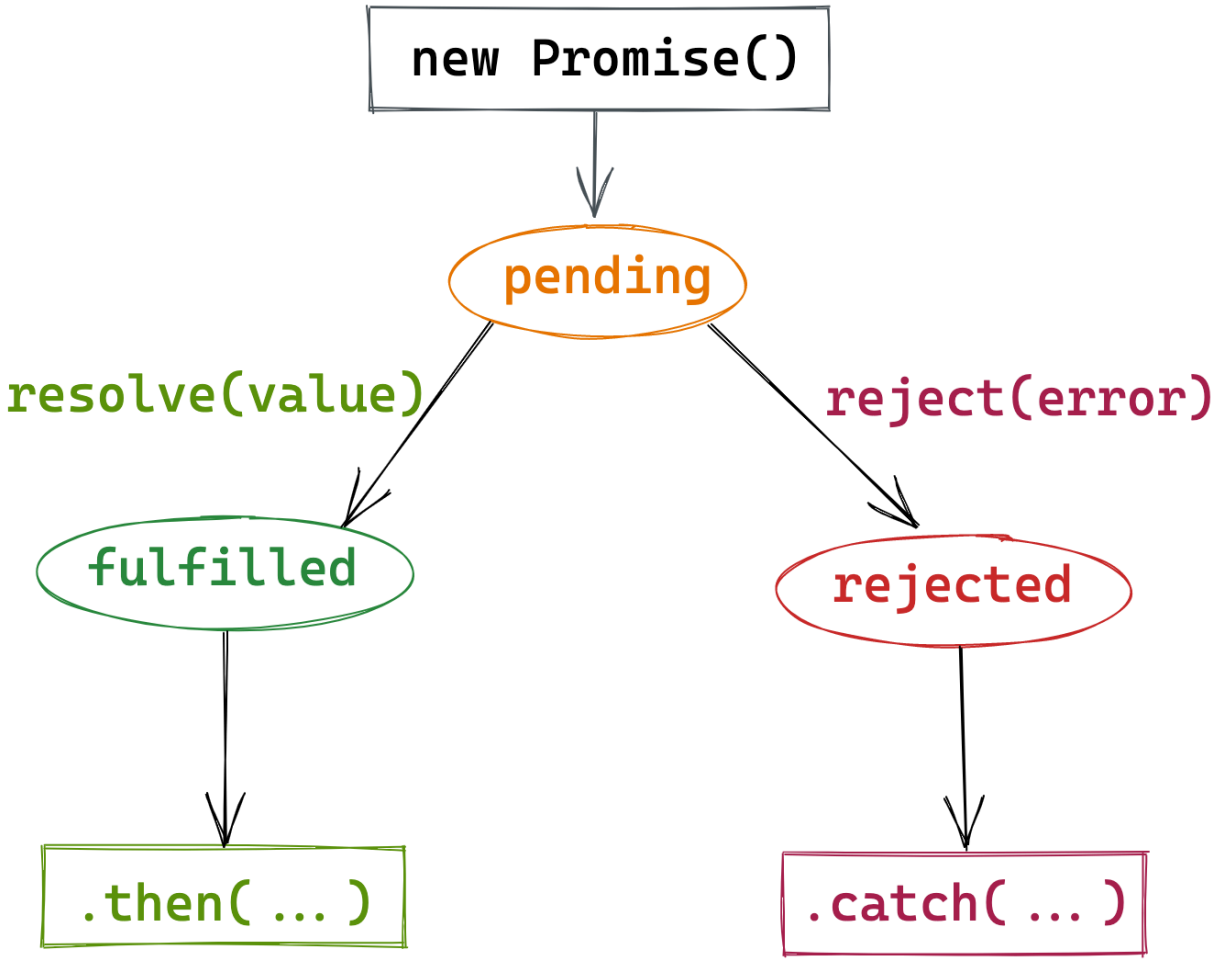
```
function executor(resolve, reject) {
  //logic goes here, call the resolve or reject to settle
the promise
}

new Promise(executor) // The promise object automatically
pass the resolve and reject arguments to the executor
function
```

Return value of a Promise

A Promise does not return a value like a traditional function does, instead it provides a state which represents the eventual completion (success or failure) of an asynchronous operation and allows you to handle the result obtained from the async operation when it's ready.

How a Promise works

When you initialize a Promise, it provides a pending state letting the JavaScript engine know that an asynchronous operation is under way and upon completion of that operation, the state changes to either fulfilled or rejected depending on the outcome of that operation, it then obtains the value/error from that operation. That value/error can be used/handled further using .then() and .catch() chaining methods.



From the diagram above, we can see that the state can change from pending to fulfilled if the async operation resolves with a value, that value is further passed to the .then() method for further operations.

The state can also change from pending to rejected if the async operation rejects with an error, that error is further passed to the .catch() method for further handling

It is important to know that the state of a Promise can only change from *pending* to *fulfilled* or from *pending* to *rejected* and not vice versa. It is also not possible for the state to change from *fulfilled* to *rejected* or vice versa.

🔹 Basic Example

```
const myPromise = new Promise((resolve, reject) => {
  let success = true;

  if (success) {
    resolve(5+3);
  } else {
    reject("Operation failed!");
  }
});
```

Here, we use the *success* variable (set to *true*) to determine if this Promise is fulfilled or rejected. In our case it'll always get fulfilled unless we change the value to false.

Consuming Promises

The previous code snippet above only changes the state of the Promise to fulfilled but we can't access or use the value (8) obtained from the resolve function yet because it is stored by the Promise object and passed to the .then() method. To use it, we call the .then() method and pass a callback function to it; it accepts the value as an argument.

```
myPromise
  .then(result => {
    console.log(result); // 8
  })
  .catch(error => {
    console.error(error);
  });
```

Here we use the callback function in the .then() method to accept the value passed by the Promised object and log it to the console. The .catch() block never gets run unless we make the Promise return a rejected state by changing the success variable to *false*

The resolve() and reject() functions can take **any** JavaScript data type including undefined (if no value is passed in). In our example, we resolved with a *number* and rejected with a *string* but we could do much more than that.

It is best practice to reject with an Error object or a plain object like:

```
reject({ status: 500, message: "Server error" }); // Plain object
//OR
reject(new Error("Something went wrong")); // Recommended: Error object
```

Chaining Promises

Sometimes a Promise can return another Promise, enabling chaining. In such cases, we can chain the .then() method to get to our desired result. A common example of this is the chaining done when fetching data using the fetch API:

```
fetch('https://api.example.com/data') // 1. Returns Promise<Response>
  .then(response ⇒ response.json())  // 2. Returns Promise<ParsedData>
  .then(data ⇒ console.log(data))    // 3. Receives the final data
  .catch(error ⇒ console.error(error)); // Handles all errors
```

When chaining promises, it is important to remember;

- Always return a Promise inside .then() to continue the chain.
- Each .then() waits for the previous Promise to resolve.
- One .catch() handles errors for the entire chain.

Nowadays, this can be done using async/await which makes working with promises and asynchronous code look and feel synchronous.

🔹 Common Promise Methods

Sometimes situations that require us to handle multiple promises simultaneously may arise for instance, fetching data from multiple endpoints. JavaScript provides some promise methods to efficiently handle those kinds of situations.

A. Promise.all()

Use Case: Parallel tasks where all must succeed (e.g., loading dashboard data).

Behavior: Fails immediately if any Promise rejects.

```
Promise.all([
  fetch('/api/users'),
  fetch('/api/products'),
  fetch('/api/orders')
])
  .then(([usersRes, productsRes, ordersRes]) => {
    return Promise.all([usersRes.json(), productsRes.json(),
ordersRes.json()]);
  })
  .then(([users, products, orders]) => {
    console.log('Dashboard data:', { users, products, orders });
  })
  .catch(error => console.error('Critical fetch failed:', error));
```

B. . Promise.allSettled()

Use Case: Run all Promises regardless of failures (e.g., analytics calls).

Behavior: Returns results for both resolved and rejected Promises.

```
Promise.allSettled([
  fetch('/api/users'),
  fetch('/api/invalid-endpoint'),  // Might fail
  fetch('/api/products')
])
  .then(results ⇒ {
    results.forEach(result ⇒ {
      if (result.status === 'fulfilled') {
        console.log('Success:', result.value);
      } else {
        console.error('Failed:', result.reason);
      }
    });
  });
```

C. Promise.any()

Use Case: Redundant requests (e.g., fallback APIs).

Behavior: Resolves with the first successful Promise; rejects only if all fail.

```
Promise.any([
  fetch('https://primary-api.com/data'),
  fetch('https://backup-api.com/data')
])
  .then(data ⇒ console.log('First successful response:',
data))
  .catch(errors ⇒ console.error('All APIs failed:',
errors));
```

🔹 Conclusion

Promises are a fundamental part of JavaScript for managing asynchronous tasks. They provide a cleaner, more readable alternative to callbacks and make it easier to handle multiple async operations.

To recap:

- A Promise is an object representing the eventual result of an asynchronous operation.
- It has three states: pending, fulfilled, or rejected.
- You can use `.then()` and `.catch()` to handle success and errors.
- Methods like `Promise.all`, `Promise.any`, and `Promise.allSettled` make it easy to work with multiple promises at once.

🔹 Connect with Me

If you found this helpful and want to learn more about JavaScript, web development, and other tech topics, feel free to connect with me on:

- **X (formerly Twitter)**: https://x.com/rolex_devv
- **GitHub**: https://github.com/rowleks
- **LinkedIn**: https://www.linkedin.com/in/rowland-momoh-b32a7a22a/
- **Portfolio**: https://rowland-momoh.netlify.app/

Let’s keep learning and growing together!